

MSDM5004

Numerical Methods and Modeling in Science

Spring 2026

Lecture 8

Prof Yang Xiang

Hong Kong University of Science and Technology

### 3. Backward Euler method

$$y'(t_i) = f(t_i, y(t_i))$$

$$y_{i+1} = y_i + hf(t_{i+1}, y_{i+1}) \quad i = 1, 2, \dots, N$$

Idea of derivation

$$\frac{y(t_{i+1}) - y(t_i)}{h} \approx y'(t_{i+1}) = f(t_{i+1}, y_{i+1})$$

$$y_i \approx y(t_i) \quad i = 1, 2, \dots, N$$

The backward Euler method is an **implicit** method

$$y_{i+1} = y_i + hf(t_{i+1}, y_{i+1})$$

In general, one needs to solve this nonlinear equation for  $y_{i+1}$

The Euler method (or forward Euler method) is an **explicit** method

$$y_{i+1} = y_i + hf(t_i, y_i)$$

$y_{i+1}$  is calculated directly from  $y_i$

Implicit methods have better stability than explicit methods  
– see more discussion in the methods for PDEs

## Linear equation

$$y' = \lambda y \quad \lambda \text{ constant}$$

Using backward Euler method,

$$y_{i+1} = y_i + h\lambda y_{i+1}$$

$$y_{i+1} = \frac{1}{1 - h\lambda} y_i$$

$$y_i = \frac{1}{(1 - h\lambda)^i} y_0$$

Note: the exact solution of this ODE is  $y(t) = e^{\lambda t} y(t_0)$

## Local error

$$e_{i+1} = y(t_{i+1}) - y_{i+1} = O(h^2)$$

## Global error

$$|y(t_i) - y_i| \leq O(h)$$

## 4. Trapezoidal method

$$y'(t_i) = f(t_i, y(t_i))$$

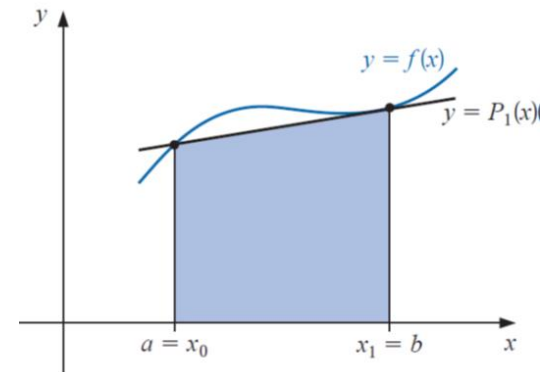
$$y_{i+1} = y_i + \frac{h}{2} [f(t_i, y_i) + f(t_{i+1}, y_{i+1})] \quad i = 1, 2, \dots, N$$

An implicit method

Idea of derivation

$$y'(t) = f(t, y(t)), \quad t \in [t_i, t_{i+1}] :$$

$$\implies y(t_{i+1}) = y(t_i) + \int_{t_i}^{t_{i+1}} f(s, y(s)) ds$$



Approximating the integral using trapezoidal rule

Other numerical methods for solving ODEs can also be derived by numerical approximation of the integral.

## Linear equation

$$y' = \lambda y \quad \lambda \text{ constant}$$

Using the trapezoidal method,

$$y_{i+1} = y_i + \frac{h}{2} [f(t_i, y_i) + f(t_{i+1}, y_{i+1})] = y_i + \frac{h}{2} (\lambda y_i + \lambda y_{i+1})$$

$$y_{i+1} = \frac{1 + \frac{h\lambda}{2}}{1 - \frac{h\lambda}{2}} y_i$$

$$y_i = \left( \frac{1 + \frac{h\lambda}{2}}{1 - \frac{h\lambda}{2}} \right)^i y_0$$

Note: the exact solution of this ODE is  $y(t) = e^{\lambda t} y(t_0)$

## Local error

$$e_{i+1} = y(t_{i+1}) - y_{i+1} = O(h^3)$$

## Global error

$$|y(t_i) - y_i| \leq O(h^2)$$

## 5. Linearization of an implicit method

$$y_{i+1} = y_i + \frac{h}{2} [f(t_i, y_i) + f(t_{i+1}, y_{i+1})]$$

$$y'(t_i) = f(t_i, y(t_i))$$



Not known yet

Using linear approximation

$$f(t_{i+1}, y_{i+1}) \approx f(t_{i+1}, y_i) + \frac{\partial f}{\partial y}(t_{i+1}, y_i)(y_{i+1} - y_i)$$

solve for  $y_{i+1}$

## 6. Runge-Kutta method

$$y'(t) = f(t, y(t)), \quad t \in [t_i, t_{i+1}]$$

### Idea

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(t_i) + \frac{h^3}{3!}y'''(t_i) + \dots$$

We hope to have a more accurate approximation of  $y(t_{i+1})$

$$y'(t_i) = f(t_i, y(t_i)) \quad \checkmark$$

$$y''(t_i) = f_t + f_y y' \big|_{t=t_i} = f_t + f_y f \big|_{t=t_i} \quad ?$$

$$f_t = \frac{\partial f}{\partial t}$$

$$y'''(t_i) = f_{tt} + f_{ty}f + (f_t + f_y f)f_y + f(f_{yt} + f_{yy}f) \big|_{t=t_i} \quad ?$$

...

We want to have a second order scheme

$$\begin{aligned} y(t_{i+1}) &= y + hy' + \frac{h^2}{2}y'' \Big|_{t=t_i} + O(h^3) \\ &= y + hf + \frac{h^2}{2}(f_t + f_y f) \Big|_{t=t_i} + O(h^3) \end{aligned}$$

Using

$$f(t + h, y + hf) = f + hf_t + hf f_y \Big|_{(t,y)} + O(h^2),$$

we have

$$y(t_{i+1}) = y + \frac{h}{2}f + \frac{h}{2}f(t + h, y + hf) \Big|_{t=t_i} + O(h^3).$$

$$y(t_{i+1}) = y + \frac{h}{2}f + \frac{h}{2}f(t + h, y + hf) \Big|_{t=t_i} + O(h^3)$$

Therefore, we have a second order scheme

$$y_{i+1} = y_i + \frac{h}{2}f(t_i, y_i) + \frac{h}{2}f(t_i + h, y_i + hf(t_i, y_i)).$$

It can be written as

$$\xi_1 = f(t_i, y_i)$$

$$\xi_2 = f(t_i + h, y_i + h\xi_1)$$

$$y_{i+1} = y_i + \frac{1}{2}h\xi_1 + \frac{1}{2}h\xi_2$$

**Local error**  $e_{i+1} = y(t_{i+1}) - y_{i+1} = O(h^3)$     **Global error**  $|y(t_i) - y_i| \leq O(h^2)$

This is a second order Runge-Kutta method.

## A general second order Runge-Kutta method

$$\xi_1 = f(t_i, y_i)$$

$$\xi_2 = f(t_i + \alpha h, y_i + \beta h \xi_1)$$

$$y_{i+1} = y_i + ah\xi_1 + bh\xi_2$$

Parameters of  $\alpha, \beta, a, b$  can be determined by Taylor expansions.

Another second order Runge-Kutta method

$$\xi_1 = f(t_i, y_i)$$

$$\xi_2 = f(t_i + \frac{1}{2}h, y_i + \frac{1}{2}h\xi_1)$$

$$y_{i+1} = y_i + h\xi_2.$$

## 4<sup>th</sup> order Runge-Kutta method

$$\xi_1 = f(t_i, y_i)$$

$$\xi_2 = f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}h\xi_1\right)$$

$$\xi_3 = f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}h\xi_2\right)$$

$$\xi_4 = f(t_i + h, y_i + h\xi_3)$$

$$y_{i+1} = y_i + \frac{1}{6}h(\xi_1 + 2\xi_2 + 2\xi_3 + \xi_4)$$

## 7. Multistep methods

$$y' = f(t, y),$$

$$\sum_{m=0}^M a_m y_{i+m} = h \sum_{m=0}^M b_m f(t_{i+m}, y_{i+m})$$

## Leapfrog method

$$y_{i+1} = y_{i-1} + 2hf(t_i, y_i) \quad \text{Second order method.}$$

## Adams-Bashforth methods

$$y_{i+M} = y_{i+M-1} + h \sum_{m=0}^{M-1} b_m f(t_{i+m}, y_{i+m}) \quad \text{M-step method.}$$

## Adams-Moulton methods

$$y_{i+M} = y_{i+M-1} + h \sum_{m=0}^M b_m f(t_{i+m}, y_{i+m})$$

## Backward differentiation formulae

$$\sum_{m=0}^M a_m y_{i+m} = hb_M f(t_{i+M}, y_{i+M})$$

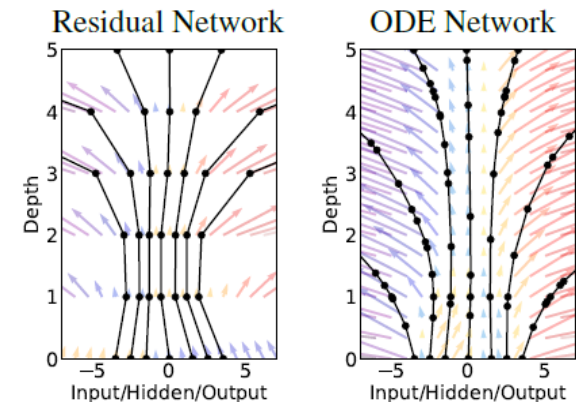
## 8. ODEs and neural networks

Neural network  $y = \mathcal{F}(\mathbf{x}, \{W_i\})$   $\mathbf{x}$ : input,  $y$ : output of next layer

ResNet [1]  $y = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$ .

Neural ODE [2]  $\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t), t, \theta)$

based on  $\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t)$



from [2]

[1] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image Recognition, CVPR 2016.

[2] R.T.Q. Chen, Y. Rubanova, J. Bettencourt, D.K. Duvenaud, Neural Ordinary Differential Equations, NeurIPS 2018.

# Chapter 10

## Numerical Methods for Partial Differential Equations (PDEs)

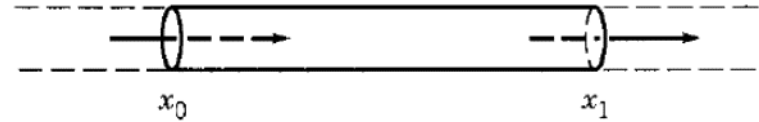
Numerical solution of partial differential equations: an introduction,  
K.W. Morton and D. Mayers, 2nd ed., Cambridge University Press, 2005.

# 1. Parabolic PDEs

## Examples: Diffusion equation in one dimension

$$u_t = ku_{xx}$$

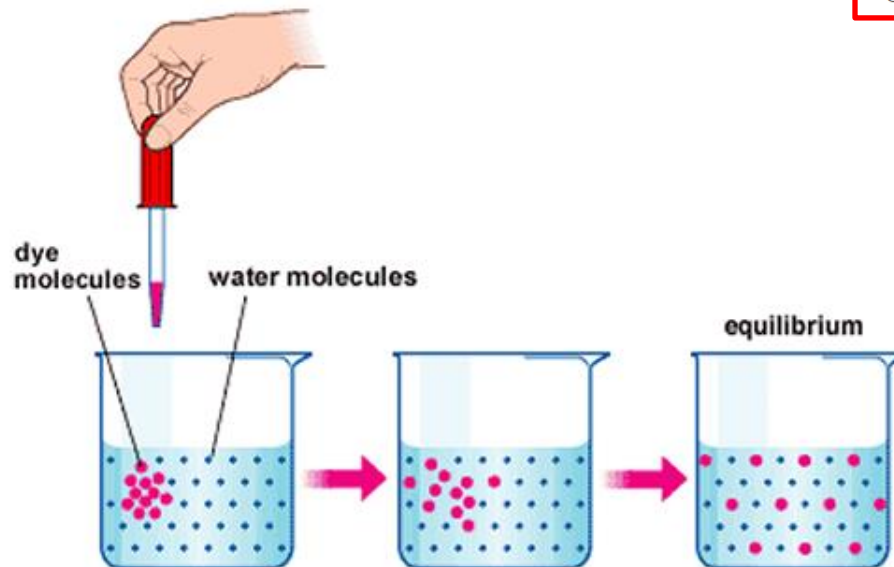
where  $k > 0$  is the diffusion constant.



## Diffusion equation in three dimensions

$$u_t = k(u_{xx} + u_{yy} + u_{zz}) = k \Delta u.$$

Example: Random motion of (dye) molecules



Laplacian operator  
in three dimensions

$$\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}$$

or  $\nabla^2$

These kind of equations can also describe the heat flow, where they are called heat equations.

# Solution of the diffusion equation

Diffusion on the whole line: initial value problem

$$u_t = ku_{xx} \quad (-\infty < x < \infty, 0 < t < \infty)$$
$$u(x, 0) = \phi(x).$$

Solution

$$u(x, t) = \frac{1}{\sqrt{4\pi kt}} \int_{-\infty}^{\infty} e^{-(x-y)^2/4kt} \phi(y) dy.$$

Once  $t > 0$ , the solution depends on the initial information at all points, i.e. the initial value at a point has an immediate effect everywhere. That means the speed of propagation is infinite.

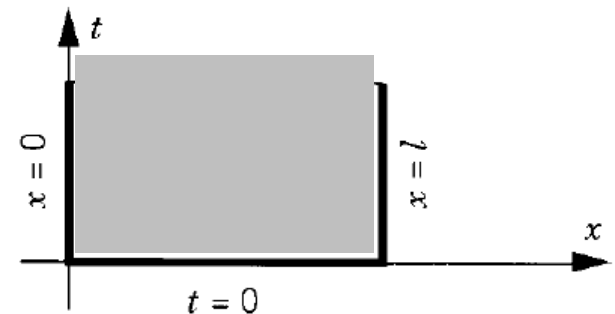
# Initial-boundary value problems

of diffusion equation

$$u_t - ku_{xx} = f(x, t) \quad \text{for } 0 < x < l \text{ and } t > 0$$

$$u(0, t) = g(t) \quad u(l, t) = h(t)$$

$$u(x, 0) = \phi(x)$$



This problem is well-posed, i.e., the solution

- 1) exists,
- 2) is unique,
- 3) continuously depend on the given data.

## 1.1. Numerical scheme

An example

$$\frac{\partial u}{\partial t}(x, t) = \frac{\partial^2 u}{\partial x^2}(x, t), \quad 0 < x < 1, \quad t > 0. \quad (1)$$

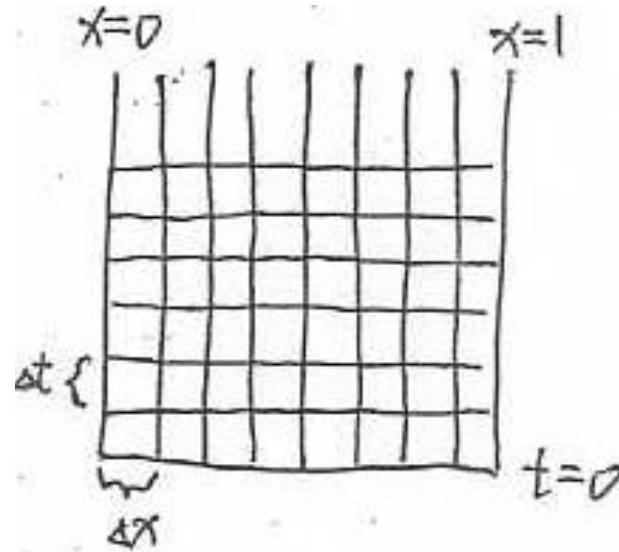
$$u(0, t) = u(1, t) = 0, \quad t > 0$$

$$u(x, 0) = u_0(x), \quad 0 \leq x \leq 1.$$

A grid (mesh)

$\Delta x$  grid constant

$\Delta t$  time step



$(x_j, t_n)$  grid point

where  $x_j = j\Delta x$ ,  $t_n = n\Delta t$ ,  $j = 0, 1, 2, \dots, J$ ,  $n = 0, 1, 2, \dots$

Seek approximation of the solution  $u(x, t)$  at grid points

$$U_j^n \approx u(x_j, t_n)$$

## Approximation of derivatives

$$\frac{\partial u}{\partial t}(x, t) = \frac{\partial^2 u}{\partial x^2}(x, t)$$

$$\frac{\partial u}{\partial t}(x_j, t_n) \approx \frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t}$$

Forward difference

$$\frac{\partial^2 u}{\partial x^2}(x_j, t_n) \approx \frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2}$$

Central difference

## Approximation of the equation

$$\frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} \approx \frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2}$$

The numerical solution satisfies numerical scheme

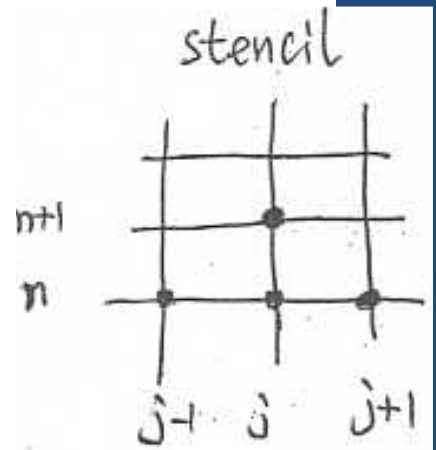
$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{(\Delta x)^2} \quad (2)$$

Or

$$U_j^{n+1} = U_j^n + \mu(U_{j+1}^n - 2U_j^n + U_{j-1}^n) \quad (3)$$

$$\mu := \frac{\Delta t}{(\Delta x)^2}$$

This is an **explicit scheme**.



$$U_j^0 = u^0(x_j), \quad j = 1, 2, \dots, J-1,$$

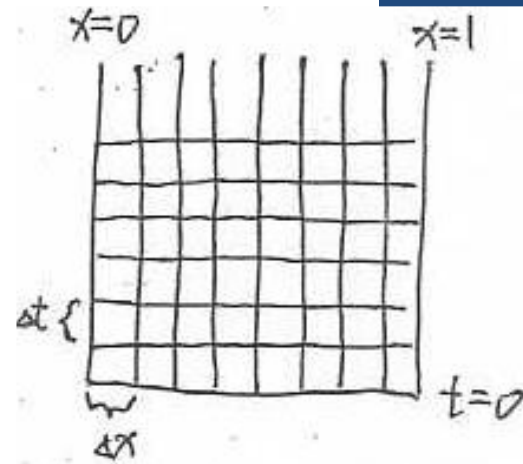
$$U_0^n = U_J^n = 0, \quad n = 0, 1, 2, \dots,$$

Compute values at time level  $t_{n+1}$  from values at time level  $t_n$

$$\begin{array}{ccccccc} U_j^0, & 0 \leq j \leq J & \Longrightarrow & U_j^1, & 0 < j < J & \Longrightarrow & U_j^2, & 0 < j < J & \Longrightarrow & \dots \\ \text{initial condition} & & & U_0^1, U_J^1 & & & U_0^2, U_J^2 & & & \end{array}$$

$$\begin{array}{ccccccc} \dots & \Longrightarrow & U_j^n, & 0 < j < J & \Longrightarrow & U_j^{n+1}, & 0 < j < J & \Longrightarrow & \dots \\ & & U_0^n, U_J^n & & & U_0^{n+1}, U_J^{n+1} & & & \end{array}$$

Explicit scheme: direct calculation  
(no need to solve linear system)



## 1.2 Truncation error and consistency

The numerical scheme 
$$\frac{U_j^{n+1} - U_j^n}{\Delta t} = \frac{U_{j+1}^n - 2U_j^n + U_{j-1}^n}{(\Delta x)^2}$$

is an approximation to the PDE 
$$\frac{\partial u}{\partial t}(x, t) = \frac{\partial^2 u}{\partial x^2}(x, t),$$

Error of this approximation: truncation error

## Truncation error

$$T(x_j, t_n) = \frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} - \frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n))}{(\Delta x)^2}$$

-- Replace the numerical solution by the exact solution in a numerical scheme.

Using Taylor expansion at  $(x_j, t_n)$

$$u(x_j, t_{n+1}) = u(x_j, t_n) + \frac{\partial u}{\partial t}(x_j, t_n)\Delta t + \frac{1}{2} \frac{\partial^2 u}{\partial t^2}(x_j, t_n)\Delta t^2 + O(\Delta t^3)$$

$$\frac{u(x_j, t_{n+1}) - u(x_j, t_n)}{\Delta t} = \frac{\partial u}{\partial t}(x_j, t_n) + \frac{1}{2} \frac{\partial^2 u}{\partial t^2}(x_j, t_n)\Delta t + O(\Delta t^2)$$

$$f(x) = O(g(x)), x \rightarrow x_0$$

$$\iff |f(x)| \leq M|g(x)| \text{ in a neighborhood of } x_0 \text{ for some constant } M$$

$$\begin{aligned}
 u(x_{j+1}, t_n) = & u(x_j, t_n) + \frac{\partial u}{\partial x}(x_j, t_n) \Delta x + \frac{1}{2} \frac{\partial^2 u}{\partial x^2}(x_j, t_n) \Delta x^2 + \frac{1}{6} \frac{\partial^3 u}{\partial x^3}(x_j, t_n) \Delta x^3 \\
 & + \frac{1}{24} \frac{\partial^4 u}{\partial x^4}(x_j, t_n) \Delta x^4 + O(\Delta x^5)
 \end{aligned}$$

$$\begin{aligned}
 u(x_{j-1}, t_n) = & u(x_j, t_n) - \frac{\partial u}{\partial x}(x_j, t_n) \Delta x + \frac{1}{2} \frac{\partial^2 u}{\partial x^2}(x_j, t_n) \Delta x^2 - \frac{1}{6} \frac{\partial^3 u}{\partial x^3}(x_j, t_n) \Delta x^3 \\
 & + \frac{1}{24} \frac{\partial^4 u}{\partial x^4}(x_j, t_n) \Delta x^4 + O(\Delta x^5)
 \end{aligned}$$

$$\frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n))}{(\Delta x)^2}$$

$$= \frac{\partial^2 u}{\partial x^2}(x_j, t_n) + \frac{1}{12} \frac{\partial^4 u}{\partial x^4}(x_j, t_n) \Delta x^2 + O(\Delta x^3)$$

Therefore

$$T(x_j, t_n) = \frac{\partial u}{\partial t}(x_j, t_n) - \frac{\partial^2 u}{\partial x^2}(x_j, t_n) + \frac{1}{2} \frac{\partial^2 u}{\partial t^2}(x_j, t_n) \Delta t - \frac{1}{12} \frac{\partial^4 u}{\partial x^4}(x_j, t_n) \Delta x^2 + O(\Delta t^2) + O(\Delta x^3)$$

Since  $u(x_j, t_n)$  is the exact solution,  $\frac{\partial u}{\partial t}(x, t) = \frac{\partial^2 u}{\partial x^2}(x, t)$ ,

$$T(x_j, t_n) = \frac{1}{2} \frac{\partial^2 u}{\partial t^2}(x_j, t_n) \Delta t - \frac{1}{12} \frac{\partial^4 u}{\partial x^4}(x_j, t_n) \Delta x^2 + O(\Delta t^2) + O(\Delta x^3).$$

- First order (accurate) in  $t$  (time)
- Second order (accurate) in  $x$  (space)

The truncation error

$$T(x_j, t_n) \rightarrow 0, \text{ as } \Delta t, \Delta x \rightarrow 0$$

We say that the numerical scheme is consistent with the PDE.

Moreover, it can be shown that

$$T(x_j, t_n) = \frac{1}{2} \frac{\partial^2 u}{\partial t^2}(x_j, \eta) \Delta t - \frac{1}{12} \frac{\partial^4 u}{\partial x^4}(\xi, t_n) (\Delta x)^2$$

$$\eta \in (t_n, t_n + \Delta t), \xi \in (x_j - \Delta x, x_j + \Delta x)$$

If  $u(x, t)$  is smooth, there exist constants  $M_{tt}$  and  $M_{xxxx}$ , such that

$$\left| \frac{\partial^2 u}{\partial t^2}(x, t) \right| \leq M_{tt}, \quad \left| \frac{\partial^4 u}{\partial x^4}(x, t) \right| \leq M_{xxxx}, \quad \text{for } 0 \leq x \leq 1, 0 \leq t \leq t_F.$$

Therefore, we have 
$$T(x_j, t_n) \leq \frac{1}{2} M_{tt} \Delta t + \frac{1}{12} M_{xxxx} (\Delta x)^2$$

$$\text{for } 0 \leq x \leq 1, 0 \leq t \leq t_F.$$